Check-in: Distribuição de Tarefas Plana vs. Hierárquica num Pequeno Cluster Emulado

Gonçalo Lourenço
Programação Paralela e Concorrente, Faculdade de Ciências, Universidade de Lisboa
fc68092@alunos.ciencias.ulisboa.pt
https://github.com/sLx-Lourenco/batchpoint

1 Introdução

Num sistema mestre–trabalhador distribuído, um coordenador reparte tarefas independentes por nós ligados em rede. Comparo duas formas de o fazer:

- **plana**: cada thread de cada nó pede ao coordenador uma tarefa de cada vez;
- **hierárquica**: um gestor em cada nó pede um lote de k tarefas e reparte-o pelas threads do nó.

A plana equilibra melhor a carga, mas paga uma ida e volta na rede por tarefa. A hierárquica paga uma por lote, mas pode deixar threads paradas quando os lotes custam tempos diferentes. Meço, num cluster de 3 nós com 2 cores cada, em que tamanhos de tarefa e desequilíbrios de carga cada uma é mais rápida. Testo três hipóteses:

- **H1**: com tarefas pequenas, a hierárquica é mais rápida;
- **H2**: com tarefas grandes e carga desequilibrada, a plana é mais rápida;
- **H3**: há um ponto de cruzamento, que se desloca para tarefas mais pequenas quando o desequilíbrio aumenta.

Implementei as duas políticas de raiz em Java 21, sobre TCP (o programa está na Fig. 4, na página 3).

2 Ambiente experimental

Torre (os 3 nós).

- AMD Ryzen 7 3700X (8 cores), 32 GB, Ubuntu Server 24.04, kernel 6.8.
- 3,6 GHz fixos; SMT, *boost* e o estado de sono C2 (400 μs) desligados.
- Cada nó é um contentor Docker 29.9 com uma JVM, preso a 2 cores: nó 1 nos cores 0–1, nó 2 nos 2–3, nó 3 nos 4–5.
- O sistema fica nos cores 6–7 (cgroups).
- Java 21 (Temurin), heap fixo de 512 MB, *Serial GC*.

Portátil (o coordenador).

- Intel Core i7-13620H, 16 GB, kernel 7.0, OpenJDK 21.0.12.
- Gera as tarefas, distribui-as, verifica os resultados e grava os dados; não calcula.
- Perfil *performance* e sono do CPU limitado a 2 μs de latência; Wi-Fi e atualizações desligados durante as medições.
- Ligado à torre por um switch Gigabit: ida e volta de 120 μs (mediana), 941 Mbit/s.

3 Como medi

Cargas. *Sintética*: um ciclo *xorshift64* sobre inteiros, sem memória, calibrado na torre (1,6755 ns por iteração). Tarefas de 10 μs, 100 μs, 1, 2, 5, 10 e 100 ms, com 2,4 s de trabalho por repetição (12 s nos 100 ms). Quatro desequilíbrios, todos com o mesmo trabalho total:

- *uniforme*: todas as tarefas iguais;
- *agrupada no início* e *no fim*: 10% das tarefas 10× mais longas, seguidas no início ou no fim da fila;
- *baralhada*: as mesmas 10%, em posições aleatórias.

Mandelbrot: 4096×4096 píxeis, até 1000 iterações, em blocos de lado 8, 32, 128 e 512. O desequilíbrio é natural: medi cada bloco num core e 15 a 22% dos blocos levam 90% do tempo.

Repetições. Corri 50 repetições seguidas em 4 configurações: a maior queda (JIT) dá-se nas 3 primeiras. Descarto 10 e meço 30 (20 nos 100 ms). Na grelha não sobra deriva: entre as primeiras e as últimas 10 repetições medidas, a variação mediana é de −0,07%.

Lote. Na hierárquica, k = tarefas/24: 24 lotes por repetição, 8 por nó. Com 20 lotes, um nó ficava com 7 e outro com 6, e a hierárquica nunca passava de 95% de eficiência. Varri k de 1 a 10 000 em 4 tamanhos: a regra fica sempre a menos de 3% do melhor lote.

Linha de base. T_1 é a mesma carga com 1 nó de 1 thread, pelo mesmo caminho (Docker, rede, coordenador); o speedup é T_1/T_6. O caminho custa 0,1 a 4,7% em relação ao cálculo puro.

Controlo. As 64 configurações correm por ordem aleatória, numa só sessão de 2 horas. Em cada repetição verifico que cada tarefa chegou uma vez e que o XOR dos resultados iguala o sequencial. Os 3 pares de cores diferem 0,26%; a torre não passou de 49 °C.

Estatística. Só 21 das 64 amostras passam no Shapiro-Wilk (que testa se os tempos seguem uma distribuição normal), por isso comparo as políticas com Mann-Whitney U (ordena os tempos das duas e vê se os de uma tendem a ficar à frente dos da outra), com correção de Holm (32 comparações: torna cada teste mais exigente consoante o número de testes feitos), e meço o efeito com o δ de Cliff e a razão das medianas.

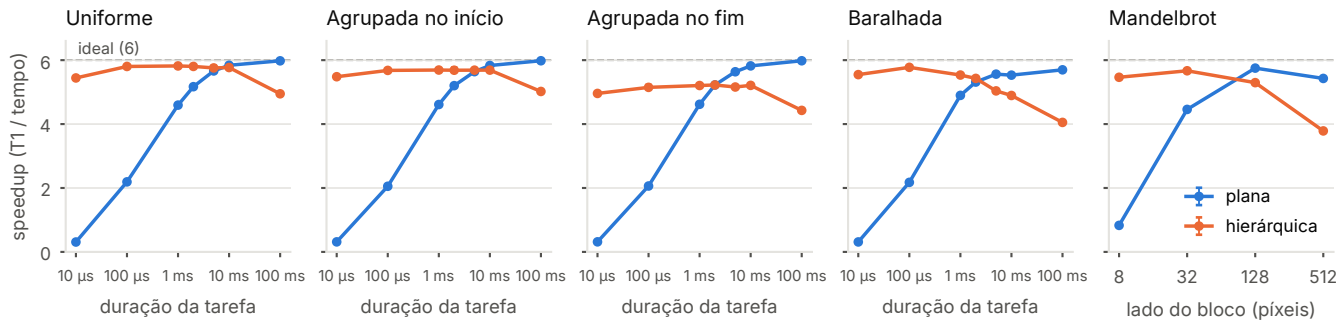


Figura 1. Speedup (T_1/T_6): mediana e IC a 95% (bootstrap; quase sempre menor do que o ponto). A carga sintética com cada desequilíbrio e o Mandelbrot; a tracejado, o ideal.

Tabela 1. Razão das medianas plana/hierárquica (> 1: a hierárquica é mais rápida). $p_{\text{Holm}} < 0{,}01$ em todas, exceto *. †: lote ímpar.

Tarefa	Uniforme	Agr. início	Agr. fim	Baralhada
10 μs	17,6	17,5	15,7	17,8
100 μs	2,65	2,77	2,50	2,65
1 ms	1,27	1,23	1,13	1,13
2 ms	1,12	1,09	1,00*	1,02
5 ms	1,02	1,01	0,92	0,91
10 ms	0,99	0,98	0,90	0,89
100 ms	0,83†	0,84†	0,74†	0,71†
Mandelbrot, lado 8 / 32 / 128 / 512:	6,58 / 1,27 / 0,92 / 0,70†			

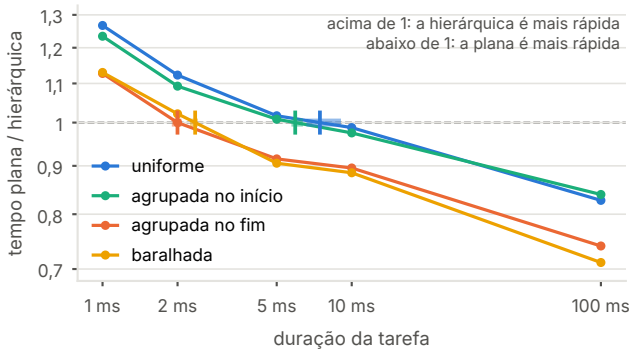


Figura 2. Razão plana/hierárquica entre 1 e 100 ms. Traço: o cruzamento (interpolado em log-log); barra: o seu IC a 95% (bootstrap).

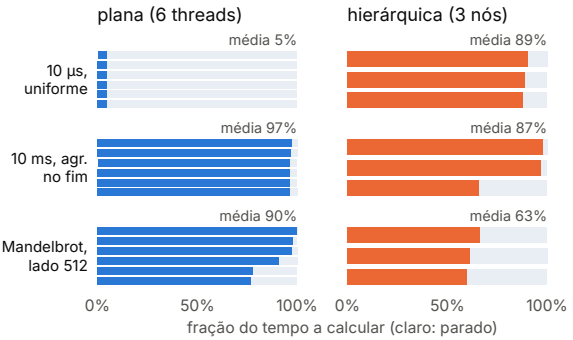


Figura 3. Fração do tempo a calcular, por thread (plana) ou por nó (hierárquica), da mais à menos ocupada.

4 Resultados

H1: confirmada (Fig. 1, Tab. 1). Com 10 μs, a hierárquica é 15,7 a 17,8× mais rápida; com 1 ms, ainda 1,13 a 1,27×. A causa é o número de mensagens: com 10 μs, a plana troca 720 000 por repetição e a hierárquica 72, para um volume parecido (4,0 vs. 2,8 MB). As threads da plana passam 95% do tempo à espera (Fig. 3).

H2: confirmada. Com 5 e 10 ms e as longas no fim ou baralhadas, a plana é 1,09 a 1,13× mais rápida ($\delta = -1$); no Mandelbrot de lado 128, 1,09×. O mecanismo vê-se na Fig. 3: com as longas no fim, os 2 últimos lotes são pesados e prendem 2 nós, enquanto o terceiro fica 34% do tempo parado. A plana reparte essas tarefas uma a uma e mantém as 6 threads a 97%. Com as longas *no início*, os lotes pesados saem primeiro e os leves preenchem o fim: as duas políticas quase empatam.

H3: confirmada (Fig. 2). O cruzamento fica nos 7,5 ms com a carga uniforme [IC 6,6–9,1], e desce para 5,9 ms com as longas no início [5,7–6,4], 2,4 ms com as baralhadas [2,2–2,4] e 2,0 ms com as longas no fim [1,9–2,1]. Os ICs dos três desequilíbrios não tocam no da uniforme.

Lote ímpar (†). Nos 100 ms, o lote de 5 tarefas para 2 threads deixa uma thread parada no fim de cada lote: 83% de eficiência mesmo na uniforme (o mecanismo da H2, isolado). O efeito é igual nos 4 desequilíbrios, por isso comparo-os entre si: face à uniforme, a hierárquica fica 1,12× mais lenta com as longas no fim e 1,22× com as baralhadas. O lado 512 tem o mesmo efeito (lote de 3).

Ameaças. Os nós partilham a cache L3; o JIT varia 1–2% entre arranques da JVM. **Próximo passo.** Política hierárquica com *work stealing* entre nós, medida na mesma grelha.

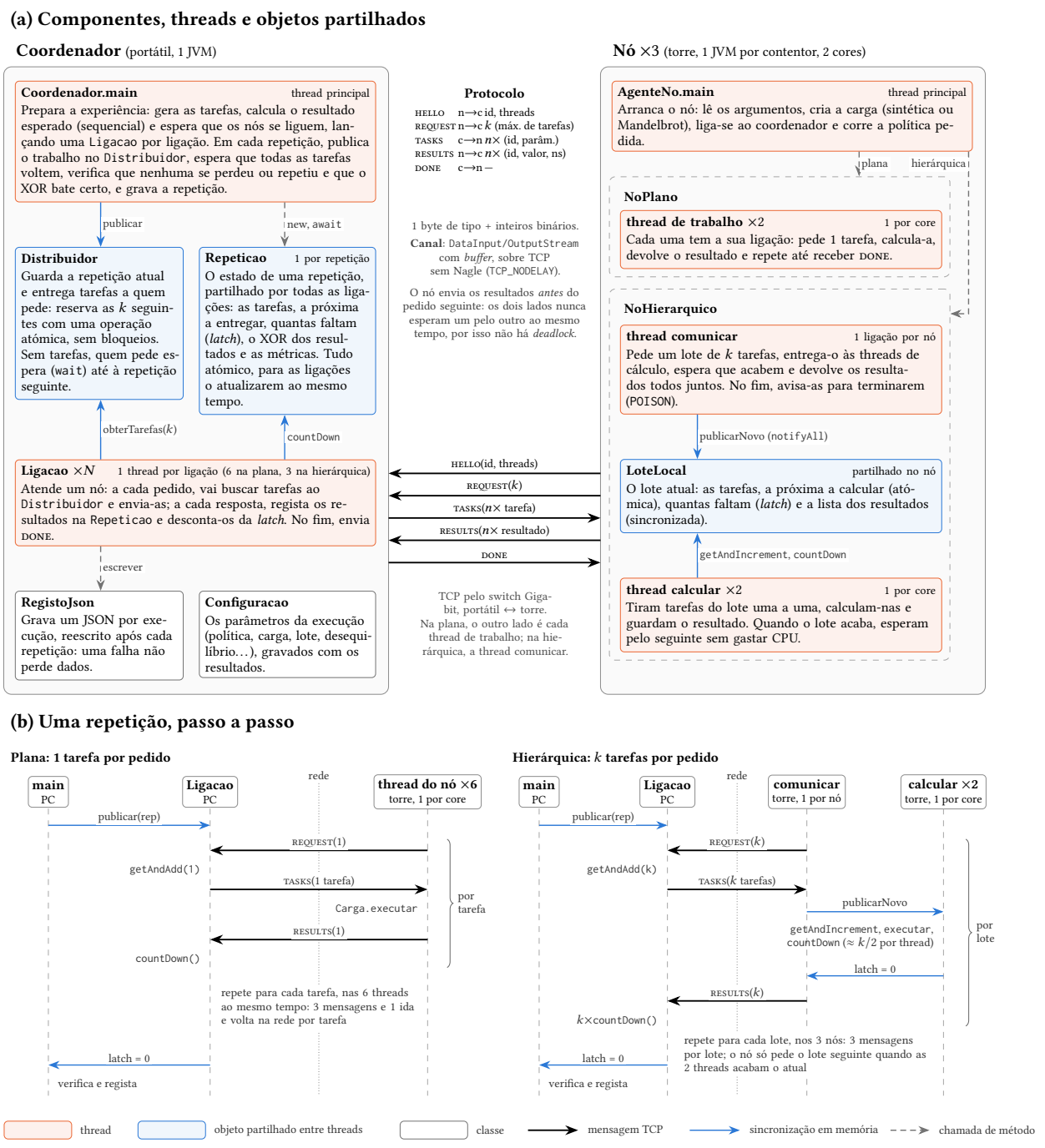


Figura 4. O programa. (a) No coordenador, a thread principal publica cada repetição num `Quadro`; cada `Ligacao` (uma thread por ligação TCP) reserva tarefas com uma operação atómica e devolve os resultados, que descontam a *latch* da repetição. No nó, a plana tem uma ligação por thread; a hierárquica tem uma thread que comunica e duas que calculam, ligadas por um `LoteLocal` partilhado. (b) A diferença entre as políticas está no número de tarefas por pedido: a plana troca 3 mensagens por tarefa, a hierárquica 3 por lote.

Uso de IA. Usei o Claude (Anthropic) no apoio à depuração do codigo Java (como prints), toda a gama de testes, nos scripts de execução e de análise e na redação.